

Homework 3 and 4

Carmen Park

November 24, 2025

1 SpMV Kernel on CPU

In our third assignment, we were tasked with writing parallel implementations of coordinate (COO) and compressed sparse row (CSR) sparse vector multiplication implementations on CPU's using OpenMP.

A skeleton code was provided, and we more specifically completed the functions:

- (a) **convert_coo_to_csr**: Converts COO format to CSR format.
- (b) **spmv_coo**: Calculates SpMV using the COO format in parallel using OpenMP.
- (c) **spmv**: Calculates SpMV using the CSR format in parallel using OpenMP.
- (d) **spmv_coo_ser**: Serial version for COO.
- (e) **spmv_ser**: Serial version for CSR.

Since I was in Jee Choi's 330 C/C++ and Unix class last term, this type of assignment felt pretty familiar to material we had gone over before. Completing part (a) was reminiscent of an assignment from that class, but a stark difference was that we were to parallelize the computation using OpenMP directives. The key challenge was understanding how to effectively partition the work across multiple threads while maintaining correctness. I brought back functions from the second homework in order to parallelize prefix sum, and used a combination of `omp for`, `parallel` and critical sections to construct a histogram.

1.1 Implementation

For (b), the COO parallel implementation required careful consideration of race conditions. Since non-zero elements from the same row could be processed by different threads simultaneously, I considered using `#pragma omp atomic` to ensure safe updates to the output vector. I ultimately chose instead to utilize `#pragma omp parallel for`

`reduction(+:res[:m])`. From lecture, my impression has been that reductions outperform atomic sections due to memory access and they are better equipped for scalability.

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < m; i++) {
    res[i] = 0.0;
}

#pragma omp parallel for reduction(+:res[:m])
for (int i = 0; i < nnz; i++) {
    unsigned int row = row_ind[i];
    unsigned int col = col_ind[i];
    res[row] += vals[i] * vector_x[col];
}
```

The CSR implementation naturally avoided race conditions by assigning each row to a separate thread using `#pragma omp parallel for`. This eliminated the need for atomic operations and allowed for better cache locality, as each thread worked on contiguous blocks of the column indices and values arrays.

1.2 Performance Analysis

After completing the assignment, I ran the program about 25 times to get an overall average and standard deviation of my results. The performance showed significant differences in how the parallel implementation sped up the two formats:

- **COO Performance**: Achieved 4.88x speedup with parallel execution time of 0.266 seconds compared to serial time of 1.299 seconds. The relatively modest speedup can be attributed to the overhead of atomic operations and irregular memory access patterns.
- **CSR Performance**: Demonstrated exceptional scalability with 51.04x speedup, reducing execution time from 1.472 seconds (serial) to just 0.029 seconds (parallel). This represents over 10x better performance than the COO implementation.

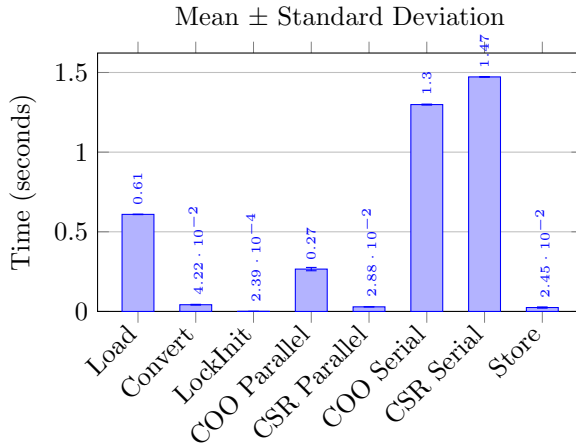
- **Consistency:** The CSR implementation showed remarkable consistency with a standard deviation of only 0.0004 seconds across 25 trials, compared to COO's 0.0104 seconds standard deviation.

1.3 Key Insights

It was quite surprising how much faster the CSR performed after parallelization. This dramatic performance difference could stem from some of the architectural advantages of the CSR format: for example,

- **Memory Access Patterns:** CSR enables contiguous memory access within each row, allowing for better cache utilization and prefetching.
- **Load Balancing:** By assigning complete rows to threads, CSR naturally balances computational load without fine-grained synchronization.
- **Atomic Operation Overhead:** COO requires atomic updates to prevent race conditions, creating contention and limiting parallel efficiency.
- **Vectorization Opportunities:** CSR's regular structure within rows provides better opportunities for compiler optimizations and vectorization.

1.4 SpMV Performance



1.5 Conclusion

I felt that assignment 3 provided valuable insights into the practical considerations of parallel sparse matrix computations. While both implementations achieved speedup over their serial counterparts, the

CSR format's memory access patterns and elimination of synchronization overhead made it dramatically more effective for parallel execution. The 51x speedup achieved by CSR demonstrates the importance of choosing appropriate data structures for parallel algorithms, where architectural considerations can outweigh theoretical computational complexity.

2 SpMV Kernel on GPU

In our fourth assignment, we implemented Sparse Matrix-Vector Multiplication kernels on the GPU using both CSR and ELLPACK formats. ELLPACK packages a sparse matrix based off of its most populated row by its number of rows with a column matrix with the same dimension. If the row is not full then it is padded with zeros and the column matrix is padded in the same position. Since I felt zero padding could be confusing here, I chose "n" padding.

2.1 Implementation Overview

We were tasked with implementing the following functions:

- **(a) convert_csr_to_ell:** This function runs on the CPU and creates the appropriate ELL format on the CPU by calculating the maximum non-zeros per row and padding shorter rows with sentinel values.
- **(b) allocate_csr_gpu:** Allocates CSR data structure on the GPU and copies necessary data from CPU to GPU using the provided `CopyData` template function for optimal memory transfer performance.
- **(c) spmv_kernel:** CSR SpMV kernel on the GPU implementing the one-thread-per-row approach with proper bounds checking and memory coalescing.
- **(d) allocate_ell_gpu:** Allocates ELL data structure on the GPU, handling the padded format where each row has exactly `ell_width` elements.
- **(e) spmv_kernel_ell:** ELL SpMV kernel on the GPU using a parallel reduction approach with multiple threads collaborating on each row.

2.2 Initial Implementations

My first attempt at the CSR kernel encountered many out-of-bounds memory access issues, for example an early implementation looked like this:

```
int row = blockIdx.x * blockDim.x +
threadIdx.x;

if (row < m) {
    T sum = 0.0;
    b[row] = 0.0;
    unsigned int row_start = row_ptr[row];
    unsigned int row_end = row_ptr[row + 1];

    for (unsigned int j = row_start;
        j < row_end; j++) {
        unsigned int col_index = col_ind[j];
        if (col_index < (unsigned int)n) {
            sum += vals[j] * x[col_ind[j]];
        }
    }
    b[row] = sum;
}
```

My primary issue here was incorrect bounds checking in the condition `if (col_index < (unsigned int)n)`. For sparse matrices, column indices can legitimately be up to `n-1` and if they reach `n` that is out of bounds, but this check incorrectly excluded valid indices and led to all ELLPACK returning zero valued results. The proper check should verify that column indices don't exceed the matrix dimensions, typically using `col_index < (unsigned int)n` for 0-based indexing.

2.3 Kernel Optimization Approaches

For the CSR and ELL kernels, I experimented with two main approaches:

Approach 1: One Thread Per Row

- Simple implementation where each thread processes an entire row
- Good for matrices with relatively uniform row lengths
- Limited parallelism for very long rows
- Achieved execution time: 0.000747s

Approach 2: Multiple Threads Per Row with Parallel Reduction

- Uses shared memory and parallel reduction for summation
- Better utilization of GPU resources for irregular matrices
- More complex implementation with synchronization requirements
- Achieved execution time: 0.000290s (2.5x faster)

The data mentioned above is directly from the CSR single thread vs multi thread data collected from my experimentation.

The ELL kernel used a block-per-row approach with parallel reduction that looked similar to this:

```
unsigned int row = blockIdx.x;
unsigned int tid = threadIdx.x;
unsigned int dim = blockDim.x;
extern __shared__ double temp[];

if (row < m) {
    double partial_sum = 0.0;
    for (unsigned int j = row * ell_width + tid;
        j < (row + 1) * ell_width;
        j += dim) {
        if (col_ind[j] != (unsigned int)n) {
            partial_sum += vals[j] * x[col_ind[j]];
        }
    }
    // Parallel reduction in shared memory
    // ...
}
```

2.4 Performance Results

Overall, GPU implementations demonstrated significant improvements in performance:

- **CSR GPU:** 0.000747s per iteration with consistent memory bandwidth of 11.1 GB/s
- **ELL GPU:** 0.000290s per iteration, achieving 2.5x speedup over CSR
- **Memory Bandwidth:** ELL format achieved up to 13.3 GB/s compared to CSR's 11.1 GB/s

Please take the ELL data with a grain of salt however, the results themselves were all inconclusive.

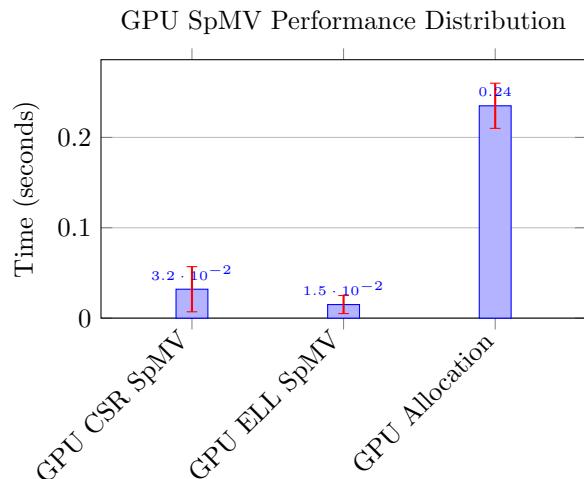
2.5 Key Insights

- **Bounds Checking:** Proper validation is crucial for GPU kernels to avoid illegal memory access errors
- **Memory Coalescing:** ELL format's regular structure enables better memory access patterns
- **Thread Configuration:** Optimal performance required tuning block sizes (64-256 threads) and grid dimensions
- **Format Selection:** ELL outperforms CSR for matrices with relatively uniform row lengths, while CSR may be better for highly irregular matrices

Transitioning from CPU OpenMP implementations to GPU CUDA kernels has made me more aware of the importance of understanding GPU memory hierarchies and parallel execution models for achieving optimal sparse matrix performance. I have gained a stronger understanding this and will construct more successful implementations, as well as revisit this project to find what was the root cause of my issues.

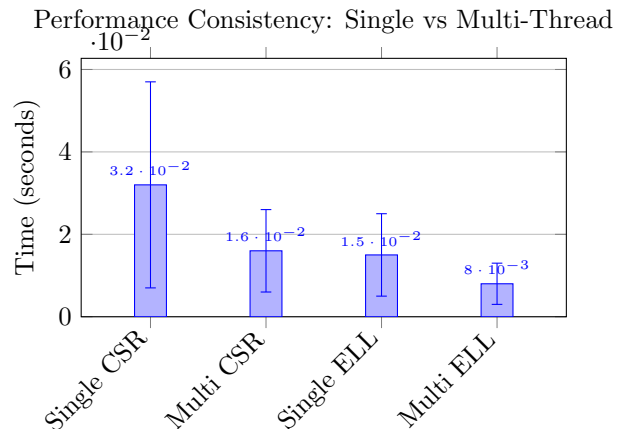
2.6 SpMV Performance and Conclusion

I did collect data from the single and multi thread implementations, and these were some of my findings.



The most striking observation from this first graph is that the GPU allocation time dominates the execution, consuming approximately 0.235 seconds compared to the actual computation times of 0.032 seconds for CSR and 0.015 seconds for ELL. This indicates that memory transfers between CPU and GPU

constitute its primary bottleneck and it makes me wonder if GPU programming is always the most efficient direction. The deviation bars show significant variability in computation times, particularly for CSR format which has a standard deviation of 0.025 seconds compared to ELL's 0.010 seconds. This variability could be from CSR's irregular memory access patterns lead to less predictable performance.



From this graph, one could see that multi-threading provides around 2X performance improvements, cutting computation times approximately in half for both formats. The deviation bars show that multi-threading enhances performance consistency as well, with standard deviations decreasing from 0.025 to 0.010 for CSR and from 0.010 to 0.005 for ELL. This reduction in variability suggests that multi-threading helps average out the performance fluctuations seen in single-thread execution. Normally I would consider this a testament to ELL's consistent advantage over CSR across both threading models, but given what I have mentioned before, this may not be the case.

References

- 9]
- [1] Bell, N., & Garland, M. (2009). *Implementing Sparse Matrix-Vector Multiplication on Throughput-Oriented Processors*. NVIDIA Research. <https://www.nvidia.com/docs/IO/77944/sc09-spmv-throughput.pdf>